Sprawozdanie z Projektu 1 Biometria

Michał Sulowski

1. Opis aplikacji

Aplikacja została napisana w języku Python. Program składa się z jednego okna, w którym użytkownik może na bieżąco obserwować wyniki działania nakładanych filtrów i operacji. Użytkownik ma również możliwość skorzystania z paska narzędzi, gdzie może wczytać obraz, zapisać obraz lub zresetować wszystkie filtry do postaci początkowej domyślnej.

W projekcie wykorzystano następujące biblioteki:

- **Tkinter:** Użyty do stworzenia głównego interfejsu aplikacji
- **Pillow:** Wykorzystany do wczytywania plików graficznych z dysku oraz konwersji macierzy wynikowych z powrotem na format wyświetlany przez interfejs Tkinter.
- **NumPy:** Wszystkie operacje na pikselach (w tym sploty) zostały zaimplementowane ręcznie przy użyciu wektoryzacji i przesunięć macierzowych.
- **Matplotlib:** Wykorzystany do stworzenia modułu analizy, generującego interaktywne wykresy projekcji oraz histogram RGB.

2. Opis zaimplementowanych metod

Wszystkie operacje na obrazach zostały zaimplementowane od podstaw. Przetwarzanie odbywa się w z góry narzuconym potoku co zapobiega konfliktom między filtrami (np. nałożeniu rozmycia na obraz już poddany binaryzacji).

Aby zapewnić odpowiednią precyzję i zapobiec przepełnieniu zmiennych obraz wejściowy jest konwertowany do macierzy zmiennoprzecinkowej `np.float32`. Dopiero na samym końcu potoku wartości są przycinane za pomocą funkcji `np.clip(..., 0, 255)` i rzutowane z powrotem na typ całkowitoliczbowy,

2.1 Operacje punktowe

Operacje te modyfikują wartość każdego piksela niezależnie od jego sąsiadów, wykorzystując zoptymalizowane, wektorowe operacje biblioteki NumPy.

- **Odcienie szarości:** Konwersja realizowana jest poprzez wydzielenie trzech kanałów RGB i przemnożenie ich przez wagi (0.299, 0.587, 0.114). Zmodyfikowane kanały są następnie składane z powrotem w trójwymiarową macierz.

- **Korekta jasności kontrastu:** Zmiana jasności to dodanie przesunięcia do całej macierzy obrazu.

- **Korekta kontrastu:** Kontrast realizowany jest poprzez rozciąganie wartości wokół środka skali (wartość 128) zgodnie ze wzorem implementacyjnym: `factor * (work_array- 128.0) + 128.0.`

- **Korekta Gamma:** Wymaga uprzedniej normalizacji wartości pikseli do przedziału [0, 1]. Następnie aplikowana jest nieliniowa funkcja potęgowa `np.power(normalized, 1.0 / gamma_val)`, a wynik jest ponownie skalowany do przedziału 0-255.

- **Negatyw:** zaimplementowano jako odwrotność matematyczną `255.0 - work_array` (wykonywaną bezwzględnie po przycięciu wartości, aby uniknąć błędów ujemnych). Binaryzacja wykorzystuje funkcję `np.where`, progując macierz szarości względem zadanego parametru i przypisując pikselom skrajne wartości 0.0 lub 255.0.

- **Binaryzacja:** wykorzystuje funkcję `np.where`, progując macierz szarości względem zadanego parametru i przypisując pikselom skrajne wartości 0.0 lub 255.0.

2.2 Filtry przestrzenne

Sploty zaimplementowano metodą wektorową. Wymagało to powiększenia obrazu, za pomocą funkcji `np.pad` z trybem `mode='edge'`, co zabezpiecza algorytm przed błędami na krawędziach obrazu.

- **Filtr uśredniający (Blur 3x3):** Zaimplementowany poprzez wycięcie 9 przesuniętych względem siebie widoków macierzy obrazu. Macierze te są do siebie dodawane, a wynik dzielony przez sumę wag (9.0), co symuluje jednoczesne nałożenie maski na wszystkie piksele. Aby uzyskać większe rozmycie, ta operacja jest wykonywana w pętli 10 razy.

- **Aproksymacja filtru Gaussa:** Zastosowano parametryczną maskę 3x3, gdzie piksel centralny zyskuje wagę b^2, a jego bezpośredni sąsiedzi wagę b. Aby filtr zachował poprawność, suma wag dzielnika jest wyliczana dynamicznie ze wzoru (b+2)^2. Parametr b jest ograniczony do wartości >=1, ponieważ wartości niższe generują artefakty zamiast wygładzenia.

- **Filtr Laplace'a:** Zaimplementowany z maską posiadającą wartość 8 w centrum i -1 w otoczeniu. Wynik splotu jest mnożony przez parametr c i dodawany do oryginalnego obrazu, co pozwala na kontrolowane wyostrzanie.

- **Detekcja krawędzi (Krzyż Robertsa i Operator Sobela):** Zaimplementowano obie metody ze względu na ich różną charakterystykę. Krzyż Robertsa wykorzystuje przesunięcia macierzy o 2x2 piksele (gradienty po skosie). Operator Sobela używa pełnego otoczenia 3x3, mnożąc widoki macierzy przez odpowiednie wagi ujemne i dodatnie w celu uzyskania gradientów Gx i Gy. W obu przypadkach ostateczna magnituda krawędzi obliczana jest wektorowo z twierdzenia Pitagorasa: `np.sqrt(Gx**2 + Gy**2)`. Sobel,

dzięki większej masce, znacznie lepiej radzi sobie z obrazami zaszumionymi.

2.3 Projekcje i Histogram

Dashboard biometryczny opiera się na analizie tablicy pikseli wygenerowanej przez główny potok przetwarzania.

- **Projekcje poziome i pionowe:** W celu uzyskania danych, gdzie ciemne piksele dają wysokie piki na wykresie, przekonwertowany do szarości obraz jest poddawany konwersji: `255.0 - gray`. Następnie za pomocą funkcji sumy zliczane są sumy wzdłuż osi X oraz osi Y. Osie projekcji są ograniczane do wymiarów wyświetlanego obrazu, dla uzyskania lepszych obserwacji.

- **Histogram:** Zaimplementowany poprzez statystyczną funkcję `np.histogram`, która zlicza częstotliwość występowania pikseli w przedziałach co 5 wartości, niezależnie dla każdego z trzech kanałów RGB, prezentując wynik w formie wykresu słupkowego.

3. Prezentacja wyników działania

Wszystkie filtry i modyfikacje wprowadzane przez aplikację są widoczne na bieżąco. Oto przykłady zastosowania wybranych metod:

Obraz oryginalny:

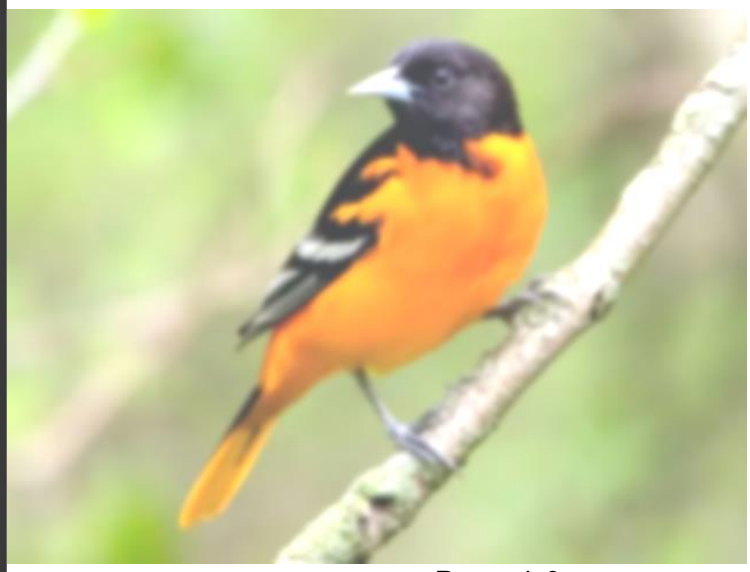
Rysunek 1

- **Binaryzacja:**

Rysunek 2

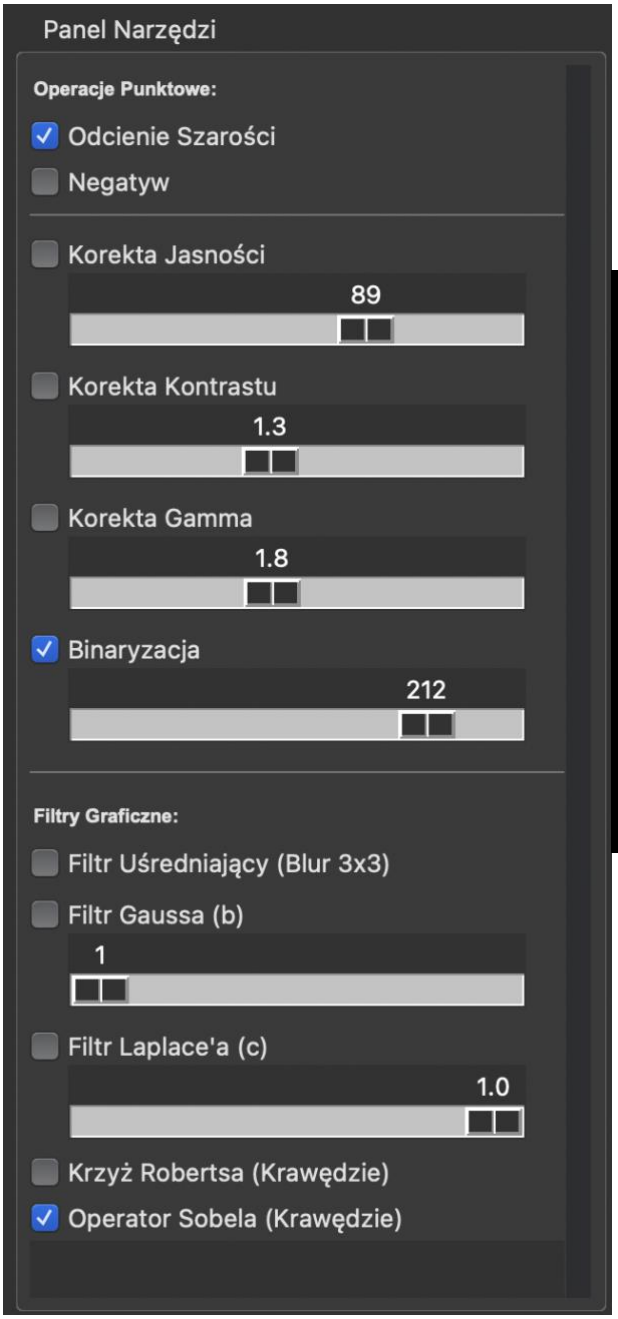
- **Zmiana jasności + blur (10 krotny):**

Obraz przetworzony



Rysunek 3

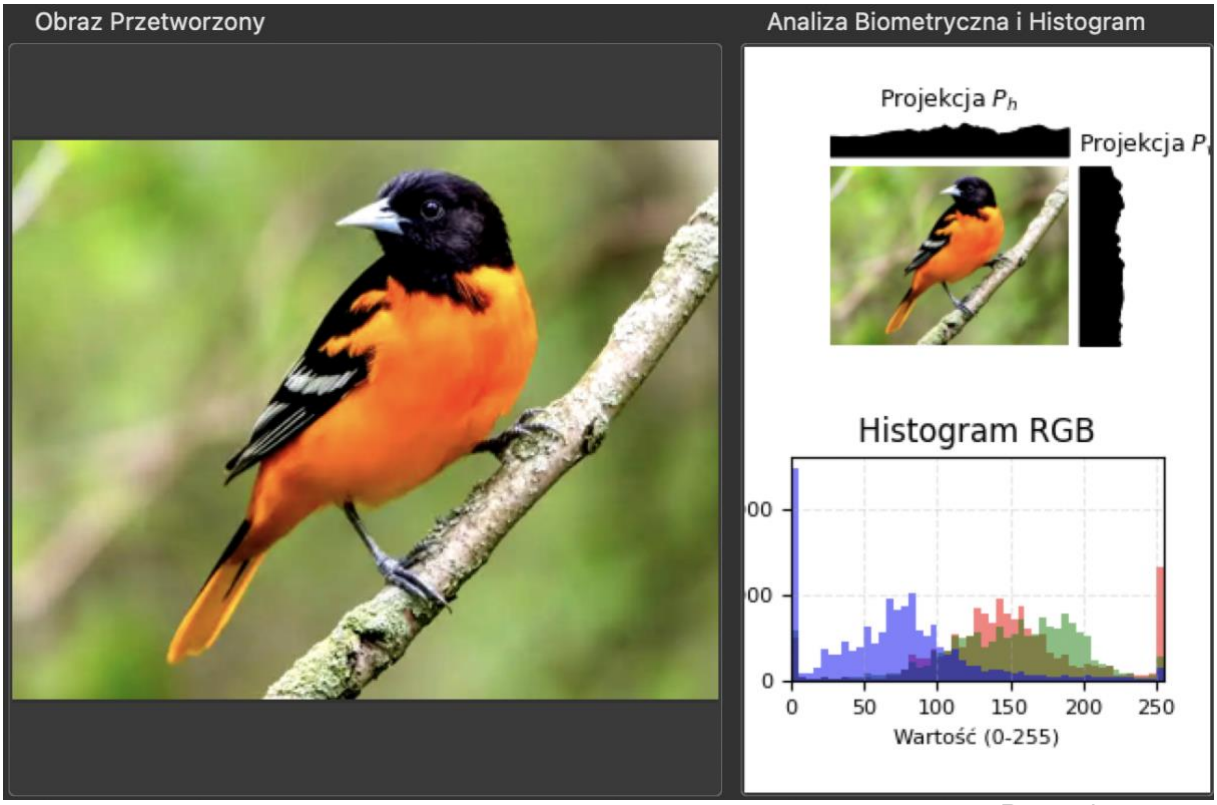
- **Odcienie szarości + operator Sobela do wykrywania krawędzi + binaryzacja:**



Obraz przetworzony

Rysunek 4

- **Zmiana kontrastu + prezentacja warstwy analitycznej:**



Rysunek 5

Wszystkie filtry i modyfikacje wprowadzane przez aplikację są widoczne na bieżąco. Jako sygnał testowy wykorzystano zdjęcie ptaka (Rysunek 1).

W pierwszej kolejności przetestowano działanie binaryzacji (Rysunek 2). Dobór odpowiedniego progu pozwolił na całkowite odcięcie tła od pierwszego planu. Następnie sprawdzono operację punktową zmiany jasności połączoną z silnym, 10-krotnym filtrem uśredniającym co poskutkowało mocnym, ale gładkim rozmyciem całego zdjęcia (Rysunek 3).

Rysunek 4 prezentuje nałożenie odcieni szarości wraz z operatorem Sobela. Widać wyraźnie, że metoda ta skutecznie wyłapała kontury gałęzi oraz piór. Na Rysunku 5 i zaprezentowano działanie warstwy analitycznej. Zmiana kontrastu wpłynęła na kształt histogramu, a wygenerowane projekcje precyzyjnie pokrywają się z fizycznymi krawędziami obiektu na zdjęciu

4. Wnioski

- **Ograniczenia parametrów wejściowych:** Podczas testowania własnej implementacji filtru Gaussa zauważyłem, że parametr wagi środkowego piksela b, nie może być zmieniany całkowicie dowolnie. Eksperymentując z wartościami poniżej 1 odkryłem, że maska splotu traci właściwości krzywej Gaussa (wagi w narożnikach stają się większe niż w centrum). Zamiast wygładzenia, filtr zaczynał generować nienaturalne artefakty i szum. Z tego powodu w ostatecznej wersji, zablokowano suwak na minimalnej wartości równej 1, co gwarantuje poprawność maski.
- **Znaczenie kolejności operacji w pipelinie:** W trakcie tworzenia programu przekonałem się, że operacje nie mogą być nakładane na obraz w losowej kolejności, a metody nie zawsze działają dobrze, jeśli nie zostaną odpowiednio przygotowane. Na przykład nałożenie binaryzacji przed rozmyciem lub detekcja krawędzi na obrazie w pełni kolorowym dawało niechciane rezultaty. Dlatego stworzony został sztywny *pipeline*, który zawsze najpierw przelicza obraz na odcienie szarości, następnie aplikuje filtry przestrzenne, modyfikacje punktowe, a binaryzację zostawia na sam koniec.